

Match3D v2: 2.5D Registration Algorithm

Technical Documentation

Match3D Development Team

May 2026

Abstract

This document describes the registration algorithms implemented in Match3D v2 for aligning 2.5D depth images (heightmaps). Unlike standard 3D point cloud registration methods, our approach treats the problem as a 2D registration in the (x, y) plane with separate handling of the z (depth) axis. This design decision addresses the fundamental scale imbalance present in 2.5D data where depth values are orders of magnitude larger than lateral coordinates.

Contents

1 Introduction

1.1 The 2.5D Data Model

A 2.5D depth image (heightmap or range image) represents a surface as a regular grid of depth measurements. Each pixel at grid position (c, r) stores a single depth value $z(r, c)$. The 3D coordinates of a point are:

$$\mathbf{p}(r, c) = \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} c \cdot \Delta_x \\ r \cdot \Delta_y \\ z(r, c) \end{pmatrix} \quad (1)$$

where Δ_x and Δ_y are the pixel sizes in meters.

1.2 The Scale Imbalance Problem

For typical dental surface scans:

- Lateral pixel size: $\Delta_x, \Delta_y \approx 30 \mu\text{m} = 3 \times 10^{-5} \text{ m}$
- Image dimensions: $\sim 500 \times 1000$ pixels
- Lateral extent: $x_{\max} \approx 0.015 \text{ m}$, $y_{\max} \approx 0.03 \text{ m}$
- Depth values: $z \approx 800\text{--}1000 \mu\text{m}$ (with arbitrary baseline offset)

This creates a **scale imbalance** where $z \gg x, y$. Standard 3D registration algorithms (ICP, Horn's method) compute rotations based on 3D Euclidean distances, causing the large z values to dominate and produce erroneous rotations.

2 Registration Pipeline Overview

The registration pipeline consists of three stages:

1. **Coarse Registration** – Initial alignment using either:
 - Center of Mass (COM) translation only
 - Landmark point pairs (2D rotation + translation + z offset)
2. **Fine Registration (Match)** – Iterative 2.5D refinement
3. **Difference Calculation** – Compute per-pixel height differences

3 Transformation Model

For 2.5D data, we restrict the transformation to:

- Rotation around the z -axis only (yaw angle α)
- Translation in all three axes (t_x, t_y, t_z)

The transformation from data coordinates to model (reference) coordinates is:

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \\ t_z \end{pmatrix} \quad (2)$$

Or in compact form:

$$\mathbf{p}' = \mathbf{R}_z(\alpha) \cdot \mathbf{p} + \mathbf{t} \quad (3)$$

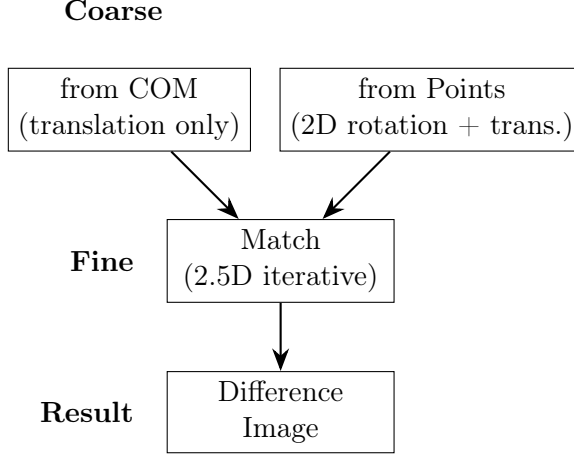


Figure 1: Registration pipeline overview

4 Coarse Registration: Center of Mass (COM)

The simplest coarse alignment translates the data image so its center of mass coincides with the reference image’s center of mass.

4.1 Algorithm

Given model image M and data image D with optional ROI masks:

$$\mathbf{c}_M = \frac{1}{|S_M|} \sum_{(r,c) \in S_M} \mathbf{p}_M(r,c), \quad \mathbf{c}_D = \frac{1}{|S_D|} \sum_{(r,c) \in S_D} \mathbf{p}_D(r,c) \quad (4)$$

where S_M and S_D are the sets of valid pixels within the ROI.

The translation is simply:

$$\mathbf{t} = \mathbf{c}_M - \mathbf{c}_D = \begin{pmatrix} t_x \\ t_y \\ t_z \end{pmatrix} \quad (5)$$

with $\alpha = 0$ (no rotation).

5 Coarse Registration: From Points (Landmark-Based)

When corresponding landmark points are available, we can compute both rotation and translation.

5.1 Problem Formulation

Given n corresponding point pairs:

- Data points: $\{\mathbf{d}_i\}_{i=1}^n$ with $\mathbf{d}_i = (x_i^d, y_i^d, z_i^d)^T$
- Reference points: $\{\mathbf{r}_i\}_{i=1}^n$ with $\mathbf{r}_i = (x_i^r, y_i^r, z_i^r)^T$

Find α and $\mathbf{t}$ that minimize:

$$E = \sum_{i=1}^n \|\mathbf{r}_i - (\mathbf{R}_z(\alpha) \cdot \mathbf{d}_i + \mathbf{t})\|^2 \quad (6)$$

5.2 2D Rotation Estimation

Key insight: For 2.5D data, we solve the rotation using only the (x, y) coordinates, ignoring z entirely.

5.2.1 Step 1: Compute 2D Centroids

$$\bar{x}^d = \frac{1}{n} \sum_{i=1}^n x_i^d, \quad \bar{y}^d = \frac{1}{n} \sum_{i=1}^n y_i^d \quad (7)$$

$$\bar{x}^r = \frac{1}{n} \sum_{i=1}^n x_i^r, \quad \bar{y}^r = \frac{1}{n} \sum_{i=1}^n y_i^r \quad (8)$$

5.2.2 Step 2: Center the Points

$$\tilde{x}_i^d = x_i^d - \bar{x}^d, \quad \tilde{y}_i^d = y_i^d - \bar{y}^d \quad (9)$$

$$\tilde{x}_i^r = x_i^r - \bar{x}^r, \quad \tilde{y}_i^r = y_i^r - \bar{y}^r \quad (10)$$

5.2.3 Step 3: Optimal 2D Rotation Angle

The optimal rotation angle that minimizes the 2D alignment error is:

$$\alpha = \arctan 2 \left(\sum_{i=1}^n (\tilde{x}_i^d \tilde{y}_i^r - \tilde{y}_i^d \tilde{x}_i^r), \sum_{i=1}^n (\tilde{x}_i^d \tilde{x}_i^r + \tilde{y}_i^d \tilde{y}_i^r) \right) \quad (11)$$

Derivation: This follows from the Procrustes problem in 2D. The optimal rotation minimizes:

$$E_{2D} = \sum_{i=1}^n \left[(\tilde{x}_i^r - \tilde{x}_i^d \cos \alpha + \tilde{y}_i^d \sin \alpha)^2 + (\tilde{y}_i^r - \tilde{x}_i^d \sin \alpha - \tilde{y}_i^d \cos \alpha)^2 \right] \quad (12)$$

Taking $\frac{\partial E_{2D}}{\partial \alpha} = 0$ yields Equation ??.

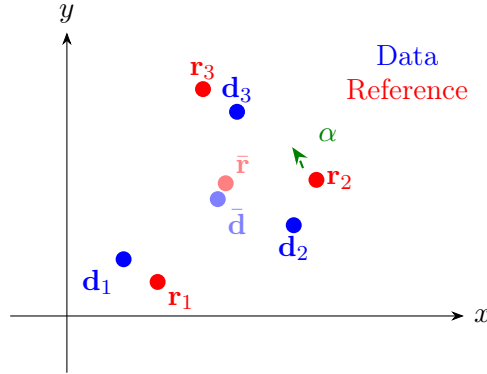


Figure 2: 2D rotation alignment: Find angle α that best aligns centered data points to reference points in the (x, y) plane.

5.3 Translation Estimation

Once α is known, the translation is computed from the centroids:

$$\begin{pmatrix} t_x \\ t_y \end{pmatrix} = \begin{pmatrix} \bar{x}^r \\ \bar{y}^r \end{pmatrix} - \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix} \begin{pmatrix} \bar{x}^d \\ \bar{y}^d \end{pmatrix} \quad (13)$$

5.4 Z-Offset Estimation (Median)

The z -translation is computed as the **median** of the z -differences at the landmark points:

$$t_z = \text{median}\{z_i^r - z_i^d\}_{i=1}^n \quad (14)$$

Why median? The median is robust to outliers. If one landmark has an erroneous z -value (e.g., due to noise or a hole in the data), it won't corrupt the entire estimate.

6 Fine Registration: 2.5D Iterative Refinement

The “Match” function performs iterative refinement of the coarse alignment. Unlike standard 3D ICP, our algorithm:

1. Finds correspondences based on (x, y) grid position, not 3D Euclidean distance
2. Computes only z -axis rotation (not full 3D rotation)
3. Uses median for z -offset (robust to outliers)

6.1 Algorithm

Algorithm 1 2.5D Iterative Refinement

Require: Model image M , Data image D , Initial transform $T_0 = (\alpha_0, t_x^0, t_y^0, t_z^0)$

Ensure: Refined transform $T^* = (\alpha^*, t_x^*, t_y^*, t_z^*)$

```

1:  $T \leftarrow T_0$ 
2: for  $k = 1$  to  $\text{maxIterations}$  do
3:   // Step 1: Find correspondences based on (x,y) position
4:    $\mathcal{C} \leftarrow \emptyset$ 
5:   for all valid model pixels  $(r_m, c_m)$  do
6:     Compute model world coordinates:  $\mathbf{p}_m = (x_m, y_m, z_m)$ 
7:     Apply inverse transform to find data position:  $(x_d, y_d) = T^{-1}(x_m, y_m)$ 
8:     Convert to data pixel:  $(r_d, c_d) = (y_d/\Delta_y, x_d/\Delta_x)$ 
9:     if  $(r_d, c_d)$  is valid in data image then
10:       $z_d \leftarrow$  bilinear interpolation of  $D$  at  $(r_d, c_d)$ 
11:       $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\mathbf{p}_m, (x_d, y_d, z_d))\}$ 
12:    end if
13:  end for
14:
15:  // Step 2: Subsample if too many correspondences
16:  if  $|\mathcal{C}| > \text{samplingLimit}$  then
17:     $\mathcal{C} \leftarrow$  random subset of size  $\text{samplingLimit}$ 
18:  end if
19:
20:  // Step 3: Compute 2D rotation update
21:  Compute centroids  $(\bar{x}_m, \bar{y}_m)$  and  $(\bar{x}_d, \bar{y}_d)$  from  $\mathcal{C}$ 
22:   $\Delta\alpha \leftarrow \arctan 2(\sum \text{cross}, \sum \text{dot})$  ▷ Eq. ??
23:
24:  // Step 4: Compute z-offset update
25:   $\Delta t_z \leftarrow \text{median}\{z_m - z_d - t_z\}$  over  $\mathcal{C}$ 
26:
27:  // Step 5: Compute translation update
28:  Update  $\alpha \leftarrow \alpha + \Delta\alpha$ 
29:  Recompute  $(t_x, t_y)$  from rotated centroids
30:   $t_z \leftarrow t_z + \Delta t_z$ 
31:
32:  // Step 6: Check convergence
33:   $\text{RMS} \leftarrow \sqrt{\frac{1}{|\mathcal{C}|} \sum (z_m - z_d - t_z)^2}$ 
34:  if relative RMS decrease  $< \text{minRMSDecrease}$  then
35:    break ▷ Converged
36:  end if
37: end for
38: return  $T = (\alpha, t_x, t_y, t_z)$ 

```

6.2 Inverse Transform for Correspondence Finding

To find where a model point (x_m, y_m) comes from in the data image, we apply the inverse transform:

$$\begin{pmatrix} x_d \\ y_d \end{pmatrix} = \begin{pmatrix} \cos \alpha & \sin \alpha \\ -\sin \alpha & \cos \alpha \end{pmatrix} \begin{pmatrix} x_m - t_x \\ y_m - t_y \end{pmatrix} \quad (15)$$

Note: $\mathbf{R}_z(-\alpha) = \mathbf{R}_z(\alpha)^T$.

6.3 Bilinear Interpolation

For sub-pixel accuracy, we use bilinear interpolation to sample the data image:

$$z_d = (1 - f_r)(1 - f_c) \cdot z_{00} + (1 - f_r)f_c \cdot z_{01} + f_r(1 - f_c) \cdot z_{10} + f_r f_c \cdot z_{11} \quad (16)$$

where $f_r = r_d - \lfloor r_d \rfloor$ and $f_c = c_d - \lfloor c_d \rfloor$ are the fractional parts.

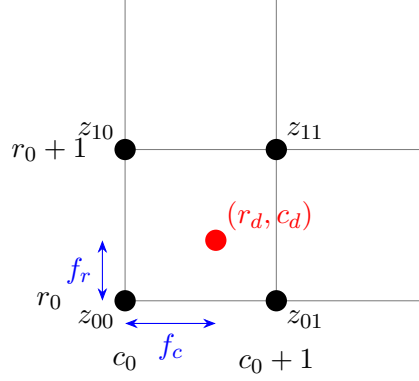


Figure 3: Bilinear interpolation: The value at (r_d, c_d) is computed as a weighted average of the four surrounding pixel values.

7 Difference Image Calculation

After registration, the difference image shows the per-pixel height difference between the aligned data and model:

$$\Delta z(r, c) = z_{\text{data}}^{\text{transformed}}(r, c) - z_{\text{model}}(r, c) \quad (17)$$

For each valid model pixel (r_m, c_m) :

1. Apply inverse transform to find corresponding data position
2. Use bilinear interpolation to get data z -value
3. Compute difference: $\Delta z = z_d + t_z - z_m$

Pixels are marked as invalid if:

- The transformed position falls outside the data image bounds
- Any of the four interpolation corners are invalid (holes)
- The pixel is outside the ROI

8 Why Not Standard 3D ICP?

Standard ICP fails for 2.5D data due to:

8.1 Scale Imbalance

With $z \approx 1000$ and $x, y \approx 0.01$, the z -axis dominates any 3D distance metric:

$$d_{3D} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2} \approx |z_1 - z_2| \quad (18)$$

8.2 Incorrect Correspondences

3D nearest-neighbor finds points that are close in the z -direction, not points that correspond to the same physical location on the surface.

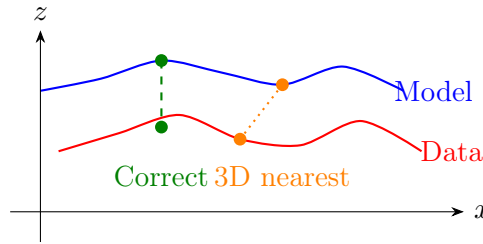


Figure 4: Correspondence problem: 3D nearest-neighbor (orange) finds wrong correspondences. The correct correspondence (green) is based on (x, y) position.

9 References

1. Kanatani, K. (1994). “Analysis of 3-D Rotation Fitting.” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 16(5), 543–549.
2. Horn, B.K.P. (1987). “Closed-form solution of absolute orientation using unit quaternions.” *Journal of the Optical Society of America A*, 4(4), 629–642.
3. Besl, P.J. and McKay, N.D. (1992). “A Method for Registration of 3-D Shapes.” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(2), 239–256.
4. Eggert, D.W., Lorusso, A., and Fisher, R.B. (1997). “Estimating 3-D rigid body transformations: a comparison of four major algorithms.” *Machine Vision and Applications*, 9(5-6), 272–290.